

Chapter 5

analyse simple algorithms and determine their efficiency using big-O notation; and

Selecting an appropriate data structure requires a mathematical framework to evaluate algorithmic efficiency independently of physical execution hardware. Measuring wall-clock runtime on a specific computer introduces confounding factors such as CPU clock frequency, compiler flags, operating system scheduling, and hardware cache structures. Algorithm analysis circumvents these empirical variables by expressing execution time and memory consumption as mathematical functions of the input size n .

This chapter establishes the principles of **asymptotic analysis**, focusing on mathematical notations used to bound performance as problem size grows toward infinity. It introduces formal definitions for **big-O notation**, Big- Ω notation, and Big- Θ notation. The chapter examines best-case, worst-case, and average-case analysis scenarios, and details formal step-counting techniques for analyzing iterative loops and simple recursive operations.

Mastering efficiency analysis enables computer scientists to predict how software implementations scale long before deployment. This mathematical grounding directly links the linear lists, trees, and hash tables examined in prior chapters to real-world application domains, laying the groundwork for analyzing complex algorithms such as data compression protocols in subsequent topics.

5.1 Principles of Algorithm Efficiency

Evaluating an algorithm requires measuring how its resource requirements scale as the size of the input n increases. The primary resources under evaluation are execution time (time complexity) and memory consumption (space complexity). Rather than measuring execution time in seconds, algorithm analysis counts primitive operations. A primitive operation is a low-level computation whose execution time is bounded by a constant, regardless of input size.

Examples of primitive operations include:

- Assigning a value to a variable.
- Performing a basic arithmetic operation (addition, subtraction, multiplication).

- Comparing two numerical values.
- Indexing into an array or dereferencing a memory reference.
- Returning a value from a function.

Let $T(n)$ represent the exact total count of primitive operations performed by an algorithm for an input of size n . For large values of n , low-order terms and constant coefficients exert negligible influence on the overall growth rate of $T(n)$. Asymptotic analysis focuses on the dominant term of $T(n)$ as n approaches infinity, isolating the underlying efficiency class of the algorithm.

5.2 Formal Asymptotic Notations

Asymptotic notation provides a mathematical language for bounding functions. Three primary notations are used to classify algorithm resource consumption: Big-O, Big-Ω, and Big-Θ.

5.2.1 Big-O Notation (Upper Bound)

Big-O notation defines an asymptotic upper bound on a function. It characterizes the mathematical ceiling of an algorithm's growth rate, ensuring that resource usage will not exceed a specific growth rate for sufficiently large inputs.

Formally, given two non-negative functions $f(n)$ and $g(n)$, $f(n) \in O(g(n))$ if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that:

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

To establish that $f(n) = 3n^2 + 5n + 8$ is $O(n^2)$, appropriate constants c and n_0 must be identified. Observing that for all $n \geq 1$:

$$5n \leq 5n^2 \quad \text{and} \quad 8 \leq 8n^2$$

Adding these inequalities yields:

$$3n^2 + 5n + 8 \leq 3n^2 + 5n^2 + 8n^2 = 16n^2$$

Setting $c = 16$ and $n_0 = 1$ demonstrates that $f(n) \leq 16n^2$ for all $n \geq 1$. Thus, $f(n) \in O(n^2)$.

5.2.2 Big-Ω Notation (Lower Bound)

Big-Ω notation defines an asymptotic lower bound on a function. It describes the minimum growth rate an algorithm guarantees for large input sizes.

Formally, $f(n) \in \Omega(g(n))$ if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that:

$$f(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0$$

For the function $f(n) = 3n^2 + 5n + 8$, observing that $5n + 8 \geq 0$ for $n \geq 1$ gives:

$$3n^2 + 5n + 8 \geq 3n^2$$

Setting $c = 3$ and $n_0 = 1$ confirms that $f(n) \in \Omega(n^2)$.

5.2.3 Big- Θ Notation (Tight Bound)

Big- Θ notation defines an asymptotically tight bound. A function $f(n)$ belongs to $\Theta(g(n))$ if and only if $f(n)$ is bounded both above and below by scaled multiples of $g(n)$.

Formally, $f(n) \in \Theta(g(n))$ if there exist positive constants $c_1 > 0$, $c_2 > 0$, and $n_0 \geq 1$ such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0$$

Because $f(n) = 3n^2 + 5n + 8$ is both $O(n^2)$ with $c_2 = 16$ and $\Omega(n^2)$ with $c_1 = 3$, it follows directly that $f(n) \in \Theta(n^2)$.

Growth Rate / Operations

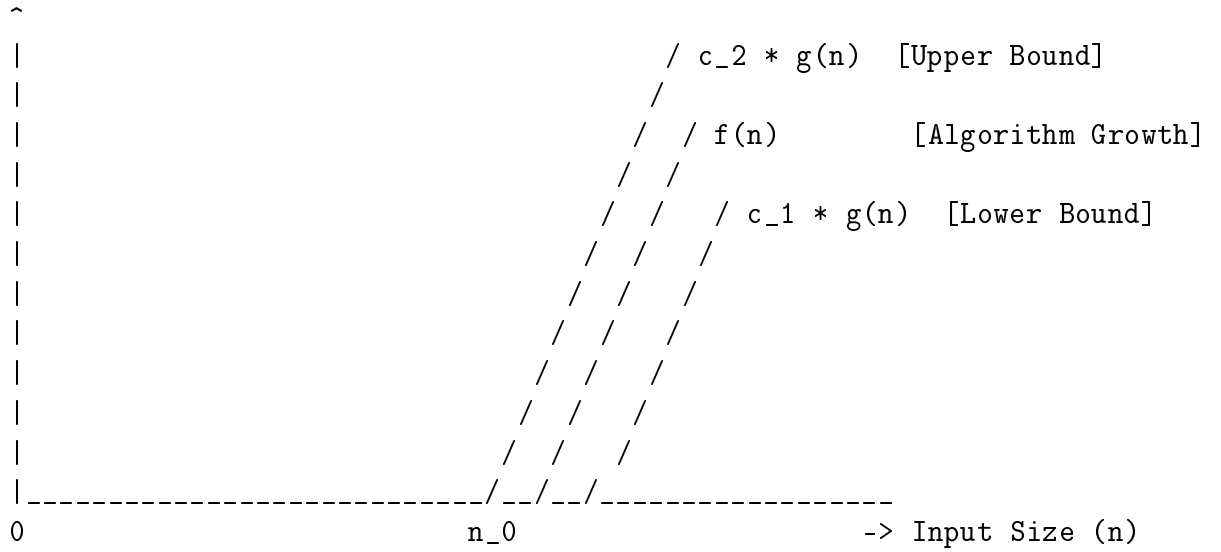


Table 5.1: Comparison of Formal Asymptotic Notations

Notation	Mathematical Inequality	Bound Type	Practical Interpretation
$O(g(n))$	$f(n) \leq c \cdot g(n)$ for $n \geq n_0$	Upper bound	Worst-case growth rate ceiling. Resource usage will not exceed this rate.
$\Omega(g(n))$	$f(n) \geq c \cdot g(n)$ for $n \geq n_0$	Lower bound	Best-case growth rate floor. Resource usage will not drop below this rate.
$\Theta(g(n))$	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for $n \geq n_0$	Tight bound	Exact growth rate matching $g(n)$ within constant factor bounds.

5.3 Analysis Scenarios: Worst-Case, Best-Case, and Average-Case

An algorithm's runtime can vary significantly based on the specific arrangement of input elements, even when input size n remains constant. Three performance scenarios are evaluated:

- **worst-case time complexity:** The maximum number of primitive operations performed over all valid inputs of size n . It provides a strict upper boundary guarantee for system safety and mission-critical design.
- **best-case time complexity:** The minimum number of primitive operations performed over all valid inputs of size n . It represents ideal conditions but rarely serves as a useful benchmark for general software performance.
- **average-case time complexity:** The expected number of primitive operations executed over all inputs of size n , weighted by their probability distribution.

Consider a linear search operation executed on an unsorted array containing n elements. The algorithm inspects each element sequentially from index 0 to index $n - 1$ until the target element is found or the array ends.

- **Best-Case:** The target element resides at index 0. The algorithm completes after 1 comparison, yielding $O(1)$ time complexity.
- **Worst-Case:** The target element is located at index $n - 1$ or is entirely absent from the array. The algorithm performs n comparisons, yielding $O(n)$ time complexity.
- **Average-Case:** Assuming the target is equally likely to be at any index from 0 to $n - 1$, the expected number of comparisons is calculated as:

$$\text{Average Comparisons} = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \left(\frac{n(n+1)}{2} \right) = \frac{n+1}{2}$$

Dropping constant factors and low-order terms establishes an average-case time complexity of $O(n)$.

5.4 Common Complexity Classes and Growth Rates

Algorithms are categorized into standard complexity classes based on their dominant growth terms. Ordering these classes from most efficient to least efficient yields:

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

The relative growth rates of these complexity functions are illustrated below:

Resource Usage $T(n)$

^
|

/ $O(2^n)$ Exponential

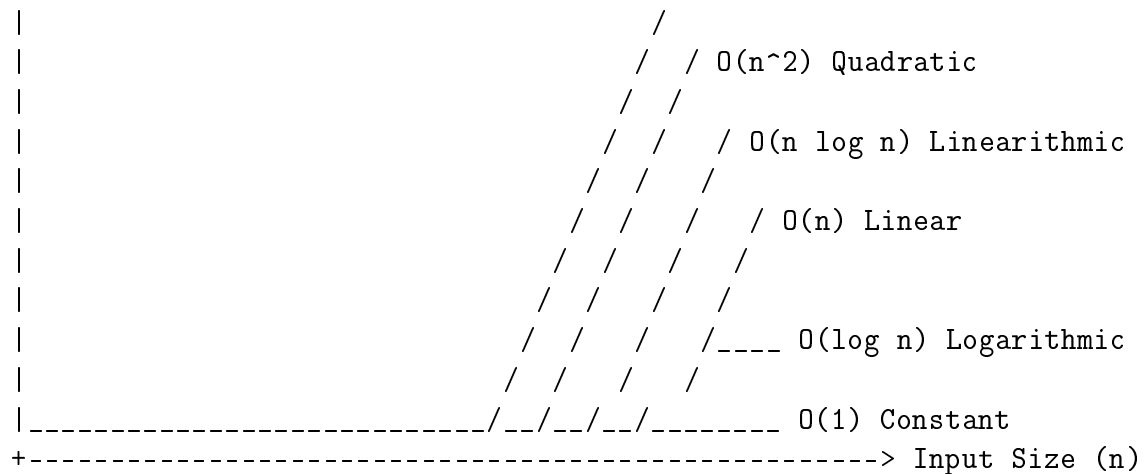


Table 5.2: Comparison of Growth Classes Across Operations Count

Complexity Class	$n = 10$	$n = 100$	$n = 1,000$	Common Data Structure Context
$O(1)$ Constant	1	1	1	array index lookup, stack push/pop
$O(\log n)$ Logarithmic	≈ 3.3	≈ 6.6	≈ 10	binary search tree lookup, binary search
$O(n)$ Linear	10	100	1,000	singly linked list traversal, linear search
$O(n \log n)$ Linearithmic	≈ 33	≈ 664	$\approx 9,966$	Merge sort, heap operations
$O(n^2)$ Quadratic	100	10,000	1,000,000	Insertion sort, nested matrix operations
$O(2^n)$ Exponential	1,024	1.27×10^{30}	1.07×10^{301}	Exhaustive recursive search

5.5 Analyzing Iterative Algorithms

Determining the time complexity of iterative algorithms requires counting operations inside loop structures and expressing total operations as algebraic sums.

5.5.1 Sequential Loops

When code blocks execute sequentially, their individual time complexities are summed. The dominant term dictates the overall upper bound:

$$T(n) = T_1(n) + T_2(n) \implies O(\max(f_1(n), f_2(n)))$$

5.5.2 Nested Loops with Independent Bounds

When loops are nested and the inner loop bound does not depend on the outer loop counter, total operations equal the product of the iteration counts.

```
def matrix_fill(matrix, n):
    for i in range(n):          # Outer loop executes n times
        for j in range(n):      # Inner loop executes n times
            matrix[i][j] = 0    # O(1) primitive assignment
```

The inner assignment runs n times for each outer loop iteration. Total primitive operations equal $n \times n = n^2$. The worst-case time complexity is $O(n^2)$.

5.5.3 Dependent Nested Loops

When loop counters interact, the inner loop bounds depend on the outer loop variable. Total execution count is determined by formulating and evaluating a summation.

Consider the following triangular loop construct:

```
def print_pairs(arr, n):
    for i in range(n):          # Outer loop runs for i = 0 to n-1
        for j in range(i + 1, n): # Inner loop runs from i + 1 to n-1
            print(arr[i], arr[j]) # Primitive O(1) operation
```

Analyzing iteration counts per outer loop step:

- When $i = 0$, inner loop executes $(n - 1)$ times.
- When $i = 1$, inner loop executes $(n - 2)$ times.
- When $i = n - 2$, inner loop executes 1 time.
- When $i = n - 1$, inner loop executes 0 times.

Total primitive operations are calculated by summing the arithmetic series:

$$T(n) = \sum_{i=0}^{n-1} (n - 1 - i) = (n - 1) + (n - 2) + \cdots + 1 + 0 = \frac{n(n - 1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

Discarding low-order terms and constant factors reveals a time complexity of $O(n^2)$.

5.5.4 Logarithmic Loops

Loops that scale their counter variable via multiplication or division in each step yield logarithmic time complexities.

```
def divide_step(n):
    count = 0
    i = n
    while i > 1:
```

```

    count += 1
    i = i // 2 # Integer division by 2
return count

```

Let k be the number of iterations executed before $i \leq 1$. The variable i takes the values $n, n/2, n/4, \dots, n/2^k$. The loop terminates when:

$$\frac{n}{2^k} \leq 1 \implies 2^k \geq n \implies k = \lceil \log_2 n \rceil$$

Because k total steps are executed, the overall time complexity is $O(\log n)$.

5.6 Analyzing Recursive Algorithms

Recursive algorithms call themselves on smaller subproblems. Analyzing their time complexity requires defining a **recurrence relation**, which expresses the overall execution time $T(n)$ in terms of $T(k)$ for smaller input sizes $k < n$.

5.6.1 Expansion Method for Recurrences

The expansion method involves repeatedly substituting a recurrence equation into itself until a recognizable pattern emerges, terminated by a base case.

Consider recursive binary search operating on a sorted array of size n . The algorithm evaluates the middle element and recursively searches either the left or right half:

$$T(n) = \begin{cases} d & \text{if } n = 1 \\ T(n/2) + c & \text{if } n > 1 \end{cases} \quad (5.1)$$

where c represents the constant time taken for middle element comparisons, and d represents base case execution.

Expanding $T(n)$ step by step:

$$T(n) = T(n/2) + c$$

Substituting $T(n/2) = T(n/4) + c$:

$$T(n) = (T(n/4) + c) + c = T(n/4) + 2c$$

Substituting $T(n/4) = T(n/8) + c$:

$$T(n) = (T(n/8) + c) + 2c = T(n/8) + 3c$$

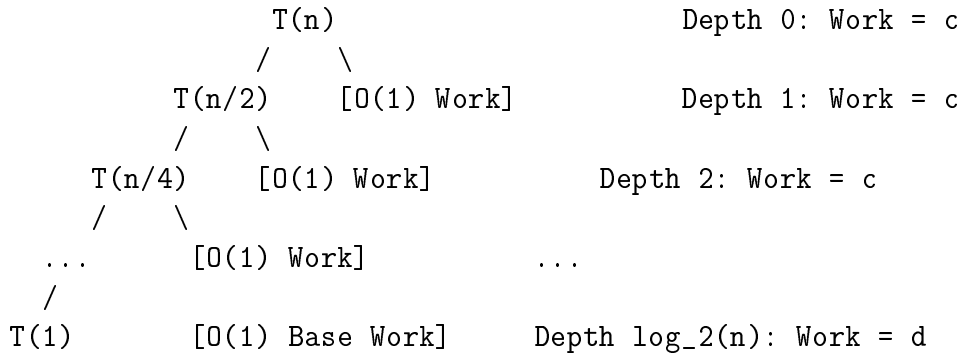
After k expansions, the general relation is:

$$T(n) = T(n/2^k) + k \cdot c$$

The expansion reaches the base case when $n/2^k = 1$, which occurs when $2^k = n$, or $k = \log_2 n$. Substituting $k = \log_2 n$ back into the expanded equation yields:

$$T(n) = T(1) + c \log_2 n = d + c \log_2 n$$

Removing constants and non-dominant terms yields a worst-case time complexity of $O(\log n)$.



Total Time Complexity: sum of work across all depths = $O(\log n)$

5.7 Space Complexity and Auxiliary Memory

Space complexity analyzes the total memory allocated by an algorithm as a function of input size n . Total space complexity comprises two distinct components:

- **Input space:** Memory occupied by the original input data structure.
- **auxiliary space:** Temporary or additional memory allocated during execution for algorithm processing, including working dynamic arrays, stack memory, or extra references.

Algorithm design prioritizes minimizing auxiliary space. For instance, an in-place sorting algorithm requires $O(1)$ auxiliary space beyond the input array, whereas non-in-place algorithms require $O(n)$ auxiliary space to allocate secondary arrays.

5.7.1 Call Stack Space in Recursive Algorithms

Recursive execution consumes auxiliary memory on the system call stack. Each active function call allocates a stack frame storing local variables, parameters, and return addresses. The auxiliary space complexity of a recursive algorithm equals the maximum depth of its call stack multiplied by the memory consumed per frame.

For instance, computing a factorial iteratively requires $O(1)$ auxiliary space:

```
def iterative_factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```


In contrast, the recursive formulation allocates n stack frames, generating $O(n)$ auxiliary space complexity:

```
def recursive_factorial(n):
    if n <= 1:
        return 1
    return n * recursive_factorial(n - 1)  # Max stack depth = n
```

Iterative Call Stack ($O(1)$ Auxiliary Space):

```
+-----+
| iterative_factorial(n): frame reused each step |
+-----+
```

Recursive Call Stack ($O(n)$ Auxiliary Space):

```
+-----+
| recursive_factorial(1)  [Depth n: Base Case]  |
+-----+
| ...                               |
+-----+
| recursive_factorial(n-1) [Depth 2]            |
+-----+
| recursive_factorial(n)  [Depth 1: Initial Call] |
+-----+
```

Table 5.3: Space Complexity Trade-offs Across Common Data Structure Operations

Data Structure / Operation	Time Complexity	Auxiliary Space	Primary Source of Space Usage
Array Indexing	$O(1)$	$O(1)$	Direct offset calculation
Linked List Traversal	$O(n)$	$O(1)$	Single pointer reference tracking
Recursion on Balanced Tree	$O(\log n)$	$O(\log n)$	Function call stack depth
Recursion on Unbalanced Tree	$O(n)$	$O(n)$	Degenerate call stack depth
hash table Lookup	$O(1)$ average	$O(n)$ total	Hash buckets and entry arrays

Evaluating both time complexity and space complexity reveals essential **time-space trade-off** principles. Choosing an optimal algorithm involves balancing the computational speed of time execution against the physical memory limits imposed by auxiliary space allocation.

5.8 Conclusion

Asymptotic analysis provides the mathematical framework for measuring algorithm performance across scaling input sizes. By classifying operations into time and space complexity bounds such as Big-O, Big-Ω, and Big-Θ, developers can rigorously evaluate algorithmic performance independently of underlying physical hardware. Understanding these analysis methods enables precise comparative evaluations across fundamental data structures.

This efficiency analysis framework serves as a core decision tool when selecting and constructing data structures for specialized software applications. The mathematical tools established in this chapter will be applied directly to evaluate domain-specific operations in the next chapter, which explores application domains such as data compression, graph indexing, and dynamic memory management.

Tutorial Questions

1. Explain why measuring the wall-clock execution time of an algorithm on a physical machine is insufficient for determining algorithm efficiency. Identify three machine-dependent factors that distort empirical measurements.
2. Formal mathematical definitions specify upper, lower, and tight performance bounds.
 - (a) State the formal mathematical definition of Big-O notation.
 - (b) Using the formal definition, prove that $f(n) = 7n^2 + 11n + 5$ is $O(n^2)$ by selecting valid positive constants c and n_0 .
3. Compare worst-case, best-case, and average-case time complexities using linear search on an array of size n as an example. Explain why worst-case time complexity is preferred for real-time safety systems.
4. Analyze the worst-case time complexity of the following code fragment by performing exact operation step counting:

```
def execute_loops(n):  
    total = 0  
    for i in range(n):  
        j = 1  
        while j < n:  
            total += arr[i] + j  
            j = j * 2  
    return total
```

5. A recursive algorithm produces the recurrence relation $T(n) = T(n - 1) + c$ for $n > 1$, with base case $T(1) = d$.
 - (a) Use the expansion method to solve for the closed-form complexity of $T(n)$.
 - (b) Identify an operation on a linear data structure that exhibits this recurrence pattern.

6. Explain the distinction between total space complexity and auxiliary space complexity. Describe a scenario where an iterative algorithm and a recursive algorithm perform identical computations with the same time complexity but different auxiliary space complexities.
7. A developer must choose between two algorithms for processing a dataset of n elements: Algorithm A has a time complexity of $O(n^2)$ with $O(1)$ auxiliary space, while Algorithm B has a time complexity of $O(n \log n)$ with $O(n)$ auxiliary space. Explain how dataset scale and hardware memory constraints should guide the selection of the appropriate algorithm.